

Synthetic Dataset Construction Pipeline

A Controlled Multi-Aspect Clinical RAG Benchmark

Master Thesis Design Note

June 6, 2026

Abstract

This document proposes a complete synthetic dataset generation pipeline for a retrieval-augmented generation (RAG) benchmark. The benchmark starts from a hidden structured plan: query facets and answer structure are defined first, atomic clinical facts are generated from that plan, facts are rendered into narrative retrieval chunks, redundant gold clusters and hard distractor clusters are created, and qrels plus gold answers are derived directly from the fact–chunk–facet mapping. MIMIC-IV is used only as a reference for terminology, condition families, comorbidity co-occurrences, demographic bins, and clinical-note style; it is not the evidence corpus being annotated after the fact. The benchmark should be presented as a controlled multi-aspect retrieval stress test in a clinical style, not as a claim that LLM-generated records are a faithful substitute for real hospital data.

Contents

1	Pipeline Core Design and Objective	3
2	Literature Map: Which Papers Matter and Why	3
3	Why the New Benchmark Should Not Be “Synthetic Discharge Notes Only”	7
4	Benchmark Objects and Data Model	7
5	Pipeline Overview	8
6	Practical MVP Implementation	9
7	Detailed Pipeline	13
7.1	Stage 1: Build a Facet Catalogue	13
7.2	Stage 2: Hidden Query Plans	16
7.3	Stage 3: Generate Structured Fact Rows	17
7.4	Stage 4: Render Fact Rows into Text Chunks via LLM	19
7.5	Stage 5: Create Role-Aware Gold Clusters	20
7.6	Stage 6: Generate Hard Distractor Chunks	21
7.7	Stage 7: Generate the Natural Query and Gold Answer via LLMs	22
7.8	Stage 8: Create Qrels Directly from the Hidden Mapping	24
7.9	Stage 9: Validate the Generated Objects	24
7.10	Stage 10: Embed and Filter by Retrieval Geometry	25
7.11	Stage 11: Evaluate Retrieval Methods	27
8	Metrics	28
9	Concrete Generation Example	29
9.1	Hidden Query Plan	29

9.2	Generated Natural Query	29
9.3	Gold Cluster Plan	30
9.4	Expected Retrieval Behavior	30
10	Recommended Ablations	30
11	Prompt and Model Strategy	31
12	Implementation Steps	31
13	Quality Report to Include in the Thesis	32
14	Threats to Validity	32
15	Final Recommended Experimental Claim	33
16	Minimal First Version	33

1 Pipeline Core Design and Objective

The experimental objective is to construct a corpus, query set, gold evidence set, and evaluation protocol in which the following hypothesis can be tested cleanly:

For broad clinical information needs that require evidence from several semantically distinct but individually redundant subtopics, a facility-location coverage objective retrieves more complete evidence than pure similarity top- k and dispersion-only MMR.

The original MIMIC-IV discharge-note pipeline did not yield a strong result because the corpus was not generated to contain this structure. Discharge notes are clinically rich, but they are noisy, patient-specific, stylistically inconsistent, and often dominated by copied sections or local admission details. In that setting, the benchmark asks a coverage method to find structure that may not actually be present in a clean way.

The new pipeline reverses the old MIMIC-IV workflow. Instead of starting from raw notes and trying to infer a benchmark, it starts from structured metadata about the dataset to be generated:

1. define query facets and answer structure;
2. generate atomic clinical facts that support each facet;
3. render those facts into narrative clinical text chunks;
4. generate redundant gold clusters and hard distractor clusters;
5. generate natural multi-aspect queries from the same hidden plan;
6. derive qrels and gold answers directly from the fact–chunk–facet mapping;
7. validate the embedding geometry before final evaluation.

This turns the dataset from a retrospective annotation problem into a prospective benchmark design problem.

2 Literature Map: Which Papers Matter and Why

The papers can be grouped by the role they should play in the new benchmark. Most generic RAG-generation papers start from an existing data source and construct QA pairs on top of it. This pipeline borrows their construction methods but changes the starting point: the real source of truth is the hidden schema that defines facts, facets, clusters, qrels, and gold answers.

Table 1: Source-to-design mapping for the proposed pipeline.

Source	Paper name	Useful idea	How it is used here
Kargupta et al. [2025]	<i>Beyond True or False</i>	Nuanced claims should be decomposed into aspect and sub-aspect hierarchies; retrieval can be aspect-discriminative.	Build a clinical facet hierarchy for each query, e.g. primary disease → subgroup → treatment duration → outcome. Each gold facet is an explicit node.

Source	Paper name	Useful idea	How it is used here
Abbasiantaeb et al. [2026]	<i>Generating Multi-Aspect Queries for Conversational Search</i>	Complex user needs can be represented as several distinct aspect queries rather than one monolithic rewrite.	Generate both a natural query and hidden subqueries, one per facet. Use subqueries for diagnostics and optional query expansion, not for the main retrieval baseline unless explicitly tested.
Sun et al. [2026]	<i>EnterpriseRAG-Bench</i>	Synthetic RAG corpora should be question-aware, cross-document coherent, noisy in realistic ways, and validated with conformance checks, multi-retriever pooling, answer facts, and retrieval-geometry diagnostics.	Build a clinical “archive” rather than isolated notes: shared condition families, care pathways, latent patient profiles, source/style scaffolds, completeness clusters, conflicting near-duplicates, anti-hallucination facts, and local-density/in-cluster/cross-cluster checks.
Gong et al. [2025]	<i>PhantomWiki</i>	On-demand synthetic worlds reduce contamination; hidden symbolic facts plus grammar-generated questions and solver-derived exhaustive answers disentangle retrieval difficulty from reasoning difficulty.	Treat the clinical corpus as a fresh generated world with fixed seeds, hidden schemas, optional grammar/logical forms for query families, and validator-derived qrels before LLM narrative rendering. Control difficulty separately through facet/reasoning depth and corpus/candidate-pool size.
Lee et al. [2025]	<i>MHTS</i>	Answer-first multi-hop construction; difficulty depends on both hop count and semantic spread of evidence.	Generate answer/evidence plans before queries. Control difficulty by number of facets, evidence clusters, and embedding distances among supporting chunks.
Huang et al. [2025]	<i>DATAGEN</i>	Attribute-guided generation, group checking, validation, and user-specified constraints improve diversity and controllability.	Use structured attributes to guide case generation. Use role-aware group checking: preserve planned redundancy inside clusters while removing accidental duplicates across facets.

Source	Paper name	Useful idea	How it is used here
Drriouich et al. [2025]	<i>Diverse And Private Synthetic Datasets Generation for RAG Evaluation</i>	Diversity agent clusters source material and samples representatives; QA curation agent produces diverse QA pairs.	Use a diversity controller that monitors cluster sizes, semantic redundancy, and coverage of the attribute grid. Privacy is less central if using fully synthetic records, but pseudonymization remains useful if MIMIC exemplars are included.
Shen et al. [2026]	<i>DRAGON</i>	A RAG benchmark should connect documents, queries, answers, clues, citations, hop counts, and answerability levels.	Store explicit mappings from query $\rightarrow$ facets $\rightarrow$ gold chunks $\rightarrow$ fact IDs. Generate sentence- or fact-level citations.
Lin et al. [2025]	<i>ScIRGen</i>	Query generation can be controlled through cognitive taxonomies and answer validation filters.	Stratify questions by cognitive operation: comparison, causal explanation, contrast, management pattern, outcome synthesis.
Rahmani et al. [2024] and Kenneweg et al. [2024]	<i>Synthetic Test Collections</i> and <i>RAGVAL</i>	LLM-generated test collections can be useful, but benchmarks must control internal-knowledge leakage and evaluation reliability.	Add a no-context or no-retrieval baseline; reject queries answerable from general medical knowledge alone.
Nadas et al. [2025], Figueira and Vaz [2022], and Li et al. [2023]	Synthetic data generation surveys and classification study	Useful synthetic data requires task-specific quality criteria, conditional/few-shot grounding, objective labels, anti-collapse checks, and authenticity reporting.	Treat diversity as a query-local structural property, not a prompt adjective. Prefer objectively labelable facets and log non-copying, prompt, model, seed, and decoding metadata.
Jadon et al. [2025]	<i>Enhancing Domain-Specific RAG</i>	Domain RAG evaluation benefits from fine-grained evidence spans and discontinuous reference spans.	Keep fact IDs and span offsets inside chunks when possible, not only chunk-level qrels.
Cosentino et al. [2025]	<i>HealthBranches</i>	Clinical QA can be generated from explicit decision pathways with full reasoning paths.	Use medical decision pathways or guideline-like templates as a source of clinically plausible treatment/outcome structure.
Gong et al. [2026]	<i>MedDialogRubrics</i>	Synthetic patient agents should be grounded in atomic medical facts; evaluation can use fine-grained rubrics.	Generate each synthetic narrative only from an atomic fact table. Validate that every narrative claim is supported by that table.

Source	Paper name	Useful idea	How it is used here
Yue et al. [2021]	<i>CliniQG4QA</i>	Diverse clinical questions can be induced by predicting/forcing question phrases.	Use question-type starters such as “How do”, “What differences”, “Which factors”, “When should”, and “Why might” to diversify query forms.
Ziletti and D’Ambrosi [2025]	<i>Generating Patient Cohorts from EHRs</i>	Complex clinical criteria should be decomposed into structured inclusion/exclusion logic and standardized concepts.	Represent queries as cohort-like logical plans over condition, demographic, comorbidity, intervention, and outcome axes.
Zafar et al. [2024]	<i>KI-MAG</i>	UMLS-style entities and triples can ground generated medical QA, even though the original system is not a retrieval-diversification benchmark.	Use concepts and relations as optional latent constraints for condition–comorbidity–treatment–outcome facts, then hide them from the evaluated retrievers.
Jin et al. [2022] and Arzideh et al. [2026]	Biomedical QA survey and clinical embedding fine-tuning	Biomedical QA often needs exact terminology, evidence grounding, and multi-document reasoning; clinical chunks can still contain many intertwined facts and require filtering.	Keep chunks narrow enough for objective facet labels, test both general and biomedical embeddings, and avoid treating embedding fine-tuning as the main thesis variable.
Qureshi et al. [2026] and Villarreal-Zegarra and Bellido-Boza [2026]	Medical synthetic-data quality and persona/profile generation	Synthetic health data should be checked for accuracy, consistency, completeness, subgroup balance, protected-attribute artifacts, and distributional drift.	Build latent patient profiles only as generation scaffolds, split by profile/query family, and report subgroup/facet balance rather than claiming population realism.
Mehandru et al. [2026]	<i>ER-Reason</i>	Clinical reasoning can be framed as evidence accumulation across workflow stages.	Optional extension: longitudinal facets, where early presentation, treatment choice, and outcome are separate evidence clusters.
Thio et al. [2026]	<i>Unlocking Electronic Health Records</i>	Hybrid structured and unstructured clinical retrieval is useful for patient journey QA.	Keep both structured metadata and narrative chunks; use metadata for generation and gold annotation, not for the main unrestricted retrieval evaluation.
Rao et al. [2026] and Montenegro et al. [2025]	<i>Scoping Review and Role of LLMs for Medical Synthetic Dataset Generation</i>	Biomedical synthetic data generation requires explicit quality, utility, and validation reporting.	Include a generation audit, plausibility checks, and a transparent threat-to-validity section.

Source	Paper name	Useful idea	How it is used here
Kim et al. [2025]	<i>Evaluating Language Models as Synthetic Data Generators</i>	A model’s ability to generate synthetic data is not identical to its ability to solve tasks; generation quality should be measured.	Compare at least two generator models or run quality checks rather than assuming the largest local model is the best generator.
Pastrián et al. [2026]	<i>Evolutionary Multi-Objective Prompt Learning</i>	Multi-objective prompt optimization can balance fidelity and diversity.	Optional later phase: automatically tune prompts against clinical plausibility, objective label consistency, and cluster diversity objectives.

3 Why the New Benchmark Should Not Be “Synthetic Discharge Notes Only”

There are three candidate strategies:

1. **Use MIMIC-IV directly.** This is realistic but too noisy and weakly controlled. The old experiment already suggests that natural discharge chunks do not reliably produce the desired cluster structure.
2. **Generate full synthetic EHR records.** This is clinically familiar, but unnecessary for the retrieval hypothesis. Full records add noise, irrelevant sections, and medical-realism obligations that distract from the core coverage experiment.
3. **Generate structured synthetic clinical evidence cards with narrative renderings.** This is the recommended approach. Each chunk is grounded in a structured fact table, but written in a discharge-note or clinical-summary style. This gives enough medical texture for realistic embeddings while preserving exact gold labels.

The third approach best fits the thesis. It lets the benchmark contain:

- controlled redundant clusters;
- explicit complementary facets;
- hard non-gold distractors;
- exact fact-level gold evidence;
- query difficulty labels;
- generation provenance.

4 Benchmark Objects and Data Model

The benchmark should produce six primary tables. These tables are more important than the natural-language text itself because they make the evaluation reproducible. A generation-audit table is useful as an implementation artifact, but it is not part of the minimal benchmark object model.

Table 2: Recommended benchmark artifacts.

Artifact	Purpose and core fields
<code>clinical_facts.parquet</code>	Hidden structured source of truth. Fields: <code>fact_id</code> , <code>condition</code> , <code>age_group</code> , <code>comorbidities</code> , <code>interventions</code> , <code>duration</code> , <code>outcomes</code> , <code>complications</code> , <code>must_mention</code> , <code>must_not_mention</code> , and <code>facet_ids</code> .
<code>chunks.parquet</code>	Retrieval corpus. Fields: <code>chunk_id</code> , <code>fact_ids</code> , <code>facet_ids</code> , <code>query_id</code> , <code>cluster_id</code> , <code>cluster_role</code> , <code>distractor_type</code> , <code>text</code> , <code>style</code> , <code>split</code> , and optional <code>span_offsets</code> .
<code>queries.parquet</code>	Natural benchmark questions. Fields: <code>query_id</code> , <code>query_text</code> , <code>condition</code> , <code>query_type</code> , <code>difficulty</code> , <code>n_facets</code> , and <code>split</code> .
<code>gold_answers.parquet</code>	Canonical answer object. Fields: <code>query_id</code> , <code>answer_text</code> , facet-level answer summaries, answer claims, supporting fact IDs, and supporting chunk IDs.
<code>query_plans.parquet</code>	Hidden plan for each query. Fields: selected facets, answer outline, <code>split</code> , <code>difficulty</code> , cluster roles, distractor families, generation constraints, optional <code>template_id</code> , optional <code>logical_form</code> , and expected corpus/candidate-pool conditions.
<code>qrels.parquet</code>	Gold evidence. Fields: query–chunk–facet labels, relevance grade, support type, fact IDs, and optional span offsets.

Optional implementation output: `generation_audit.parquet`, containing prompt hashes, model versions, decoding parameters, seeds, validators, rejection reasons, non-copying checks, and schema versions.

5 Pipeline Overview

The full pipeline follows the same eleven stages as the reduced version:

1. Build a facet catalogue.
2. Generate hidden query plans.
3. Generate structured fact rows.
4. Render fact rows into text chunks via LLM.
5. Create role-aware gold clusters.
6. Generate hard distractor chunks.
7. Generate the natural query and gold answer via LLMs.
8. Create qrels directly from the hidden mapping.
9. Validate the generated objects.
10. Embed and filter by retrieval geometry.
11. Evaluate retrieval methods.

The key methodological shift is that the generation pipeline owns the gold labels. Gold evidence should not be reconstructed after the fact by asking an LLM to read arbitrary chunks. The LLM

may validate, refine, or reject data, but the authoritative label should come from the hidden structured plan.

6 Practical MVP Implementation

The first implementation should be deliberately narrow. Do not start with Prolog, agentic retrieval, large archive scaffolds, conflicting-note resolution, or information-not-found questions. Those are later extensions. The MVP should prove the core retrieval-diversification hypothesis with a small, deterministic generator.

MVP scope.

- 4 conditions;
- 2 query templates: subgroup comparison and outcome synthesis;
- 4 facets per query;
- 10 gold chunks per facet;
- 30 hard distractors per query;
- 100–150 queries;
- one embedding model;
- top- k , MMR, and facility-location only.

Implementation rule. Use ordinary Python dataframes and deterministic sampling. Treat the “logical form” as a JSON object stored in `query_plans.parquet`; do not implement a formal logic solver. The rule-check layer is just pandas filters over structured columns.

Concrete files to implement.

Table 3: Minimal implementation modules.

Module	Responsibility
<code>ontology.yaml</code>	Manual catalogue of conditions, subgroup values, treatment-duration bins, rehab-outcome bins, and allowed distractor types.
<code>01_plans.py</code>	Deterministically sample condition, two subgroups, two axes, four facet IDs, cluster sizes, and split. Writes <code>query_plans.parquet</code> .
<code>02_facts.py</code>	Expand every plan into structured gold and distractor fact rows. Writes <code>clinical_facts.parquet</code> .
<code>03_chunks.py</code>	Render each fact row into a deterministic draft, optionally ask an LLM to rewrite it, then validate required/forbidden strings. Writes <code>chunks.parquet</code> .
<code>04_queries.py</code>	Turn each plan into one natural query and one canonical answer using the structured facts. Writes <code>queries.parquet</code> and <code>gold_answers.parquet</code> .
<code>05_qrels.py</code>	Join plans, facts, and chunks. Gold facts become positive qrels; distractors become grade 0. Writes <code>qrels.parquet</code> .
<code>06_embed.py</code>	Embed chunk text and query text. Writes a vector store or <code>embeddings.parquet</code> .

Module	Responsibility
07_geometry.py	Compute top- N pools, facet presence, top- k dominance, cluster separation, and distractor proximity. Writes <code>geometry_stats.parquet</code> .
08_evaluate.py	Run top- k , MMR, and facility-location and compute facet coverage metrics. Writes <code>evaluation_results.parquet</code> .

Minimal ontology shape.

```

conditions:
  encephalitis_myelitis:
    display: "encephalitis or myelitis"
    terms: ["encephalitis", "myelitis", "acyclovir", "neurologic deficit"]
    treatments: ["acyclovir", "corticosteroids", "supportive care"]
  pneumonia:
    display: "pneumonia"
    terms: ["pneumonia", "antibiotics", "oxygen requirement"]
    treatments: ["ceftriaxone", "azithromycin", "oxygen therapy"]

subgroups:
  age_over_75:
    axis: "demographic"
    label: "patients older than 75"
    field: "age_group"
    value: "over_75"
  uncomplicated_diabetes:
    axis: "comorbidity"
    label: "patients with uncomplicated diabetes"
    field: "comorbidity"
    value: "uncomplicated_diabetes"

clinical_axes:
  treatment_duration:
    bins: ["short", "standard", "prolonged"]
  rehab_outcome:
    bins: ["home_rehab", "inpatient_rehab", "persistent_deficit"]

```

Stage 1 implementation. Write `ontology.yaml` by hand. MIMIC-IV can help choose condition names and common comorbidities, but do not mine MIMIC chunks for gold evidence. For the MVP, use only two clinical axes: `treatment_duration` and `rehab_outcome`. This makes every comparison query produce exactly four facets: subgroup A duration, subgroup A outcome, subgroup B duration, subgroup B outcome.

Stage 2 implementation. Generate plans with nested loops, not with an LLM:

```

for condition in selected_conditions:
  for subgroup_a, subgroup_b in subgroup_pairs:
    axes = ["treatment_duration", "rehab_outcome"]
    create one query plan with 4 facets:
      F1 = condition + subgroup_a + treatment_duration
      F2 = condition + subgroup_a + rehab_outcome
      F3 = condition + subgroup_b + treatment_duration
      F4 = condition + subgroup_b + rehab_outcome

```

Set `dominant_facet=F1` for half the plans and rotate it for the other half. Use `gold_chunks=18` for the dominant facet and `gold_chunks=8` for the other facets. This directly creates the top- k redundancy failure mode.

Stage 3 implementation. Generate structured facts with templates and random draws from allowed bins. Do not call the LLM here unless needed. A fact row should have one intended facet. If one row supports both duration and outcome, split it into two rows for the MVP. This keeps qrels unambiguous.

```
fact = {
  "query_id": qid,
  "fact_id": fact_id,
  "facet_id": facet_id,
  "cluster_id": cluster_id,
  "cluster_role": "dominant_gold",
  "condition": condition,
  "subgroup_id": subgroup_id,
  "axis": "treatment_duration",
  "value": "prolonged",
  "duration_days": 14,
  "rehab_outcome": null,
  "is_gold": true,
  "distractor_type": null,
  "must_mention": ["encephalitis", "older than 75", "14-day course"],
  "must_not_mention": ["uncomplicated diabetes"]
}
```

Stage 4 implementation. Start with deterministic text templates. Add LLM rewriting only after the deterministic version works end to end.

```
Duration template:
"The patient was treated for {condition_text}. The record describes
{subgroup_text}. Treatment included {treatment} for {duration_days} days,
which was categorized as {duration_bin} for this synthetic benchmark."

Rehab template:
"The patient was treated for {condition_text}. The record describes
{subgroup_text}. At discharge, the rehabilitation outcome was {outcome_text}."
```

Validation can be simple string checks at first: every `must_mention` phrase must appear and no `must_not_mention` phrase may appear. Store failed rows in `generation_rejects.parquet`.

Stage 5 implementation. Clusters are created by row counts, not clustering algorithms. If a facet has target size 10, generate 10 fact rows and 10 chunks with the same `cluster_id`. Use wording variation through small template variants, different synthetic ages, different duration values within the same bin, and different rehab descriptions within the same outcome bin.

Stage 6 implementation. Generate exactly three distractor families per query:

1. `same_condition_wrong_subgroup`: same condition and same clinical vocabulary, but subgroup is neither A nor B;
2. `same_subgroup_wrong_condition`: same subgroup and axis, but a different condition;
3. `same_axis_wrong_condition`: same duration or rehab wording, but wrong condition and wrong subgroup.

Do not implement misfiled notes, near-duplicate contradictions, or information-not-found in the MVP. They add ambiguity before the basic coverage experiment is proven.

Stage 7 implementation. Use one deterministic query template first:

```
For patients diagnosed with {condition_display}, how do {axis_1_label}
and {axis_2_label} differ between {subgroup_a_label} and {subgroup_b_label}?
```

The answer can also be deterministic at first:

```
In {subgroup_a_label}, the synthetic corpus shows {facet_F1_summary}
for treatment duration and {facet_F2_summary} for rehabilitation outcome.
In {subgroup_b_label}, it shows {facet_F3_summary} for treatment duration
and {facet_F4_summary} for rehabilitation outcome.
```

After retrieval metrics work, ask an LLM to paraphrase only the query text and answer text. Keep `query_plans.parquet`, `clinical_facts.parquet`, and `qrels.parquet` unchanged.

Stage 8 implementation. Qrels are just a dataframe join:

```
qrels = chunks[["query_id", "chunk_id", "facet_id", "fact_id", "is_gold"]]
qrels["relevance_grade"] = qrels["is_gold"].astype(int)
```

For the MVP, every gold chunk has exactly one `facet_id`. That makes Facet Coverage@k trivial to compute and debug.

Stage 9 implementation. Use deterministic validators before using LLM judges:

- each query has exactly 4 facets;
- each facet has at least 8 gold chunks;
- each chunk has exactly one facet;
- every chunk has non-empty text and 50–180 words;
- every `must_mention` string appears in the chunk;
- no `must_not_mention` string appears in the chunk;
- each query has at least 20 distractors.

Stage 10 implementation. Use these concrete keep rules for the first run. Tune them later, but do not leave them vague:

- candidate pool size: $N = 300$ for query-local, $N = 1000$ for full-corpus;
- each facet must have at least one gold chunk in top- N ;
- top-10 similarity baseline must put at least 5 of 10 chunks in one facet for the query to be coverage-sensitive;
- mean in-facet similarity must be greater than mean cross-facet similarity by at least 0.03;
- at least 10 distractors must appear in top- N .

If too few queries pass, adjust generation, not the retrieval method: increase dominant-cluster size, make complementary clusters use fewer exact query terms, or make distractors closer.

Stage 11 implementation. Run only:

- top- k ;
- MMR with $\lambda \in \{0.3, 0.5, 0.7\}$;
- facility-location with $\lambda \in \{0.3, 0.5, 0.7\}$.

Primary metrics for the first table: Facet Coverage@5/10, Gold Precision@5/10, Distractor Rate@5/10, and Dominant-Cluster Concentration@5/10. Add answer generation later.

7 Detailed Pipeline

7.1 Stage 1: Build a Facet Catalogue

Goal. Define the benchmark scope, split policy, and clinical facet catalogue before generating any text.

Recommended claim. The benchmark evaluates retrieval diversity methods under controlled multi-aspect clinical information needs. It is not a medical decision-support dataset and should not be used to infer clinical policy.

Design choices. Use a clinical style because it naturally supports multi-aspect queries:

- disease plus comorbidity;
- disease plus demographic subgroup;
- treatment plus complication;
- intervention plus duration;
- outcome plus discharge disposition;
- disease trajectory over time.

Clinical archive scaffolding. Borrow the scaffolding idea from EnterpriseRAG-Bench [Sun et al., 2026]: generated documents should share a common synthetic reality rather than being independent one-off notes. Before writing chunks, define a small set of human-reviewed artifacts:

1. condition-family overview: target diseases, terminology, and broad management patterns;
2. care-path or guideline-like templates: plausible interventions, durations, outcomes, complications, and follow-up paths;
3. latent patient/profile catalogue: synthetic demographics, comorbidities, encounter identifiers, and split assignment;
4. source and note-style structure: discharge summary, progress note, rehabilitation consult, follow-up note, medication-management note;
5. style-agent instructions: what each source type may contain, what fields are mandatory, and which facts must never be invented.

These artifacts are not exposed to retrieval systems. They exist to keep the corpus coherent across chunks, while the query-local hidden plan still controls the exact facets and gold labels.

On-demand generation stance. PhantomWiki argues for publishing a generation procedure rather than relying on a single static dataset instance [Gong et al., 2025]. For this thesis, the practical version is to store fixed seeds and schema versions for the reported benchmark, while keeping the pipeline capable of regenerating fresh validation/test instances. This reduces leakage risk and makes it possible to test whether the facility-location effect survives across multiple generated worlds.

Recommended scale. For a first thesis-quality benchmark:

- 4–8 primary conditions;
- 8–15 attribute axes;
- 400–800 queries total;
- 20,000–80,000 chunks;
- 3–5 gold facets per query;
- 5–20 gold chunks per facet;
- 10–50 distractor chunks per query in the top- N candidate pool.

Splits. Use two reported splits:

- **validation:** used after the generation pipeline is fixed; choose retrieval hyperparameters, especially MMR and facility-location λ values;
- **test:** final locked evaluation.

Do not tune filtering thresholds or retrieval hyperparameters on the test split. Split at the `query_family` or latent-profile level, not only at the row level: surface variants of the same query plan, chunks generated from the same patient profile, and near-duplicate distractor templates should stay in the same split. This prevents leakage where validation and test differ only by wording.

MIMIC-IV and optional clinical scaffolds. MIMIC-IV can be used as a reference for terminology, frequent conditions, comorbidity distributions, age groups, and note style. It should not be treated as the corpus from which gold labels are discovered.

There are four viable ways to seed the catalogue.

Option A: MIMIC-seeded synthetic benchmark. Use MIMIC-IV only to estimate plausible distributions and wording:

- frequent conditions;
- common comorbidities;
- age distributions;
- common treatments;
- discharge-note style;
- typical section headers.

Then generate a new synthetic corpus from the structured schema. This is the recommended default because it gives realism without surrendering control.

Option B: Guideline or decision-pathway seeded benchmark. Use decision pathways, guideline-like rules, or medical educational material to define treatment and outcome structures. This follows the HealthBranches logic of grounding questions in explicit decision paths [Cosentino et al., 2025]. It is useful when you want the clinical facts to be medically coherent without using patient-level EHR data.

Option C: Fully synthetic from scratch. Define a hand-built ontology and ask the LLM to populate it. This maximizes control but requires stronger validation because clinical plausibility depends heavily on the generator.

Option D: Concept- or KG-seeded benchmark. Use UMLS-style concepts, relation triples, or normalized clinical vocabularies as latent constraints for facts. This borrows the useful part of KI-MAG [Zafar et al., 2024]: structured medical knowledge can constrain terminology and relations. The graph or concept layer should be used for generation and auditing only, not as an evaluation-time retrieval shortcut.

Recommendation. Use Option A plus a small amount of Option B and, where available, light Option D concept grounding. MIMIC supplies style and distributions; decision pathways supply management logic; concept grounding reduces terminology drift. The final corpus is still generated from the hidden schema.

Facet catalogue.

Purpose. The facet ontology defines which aspects can appear in queries and which clusters should exist in the corpus. This is inspired by ClaimSpect’s hierarchical aspect decomposition [Kargupta et al., 2025] and by cohort-style criteria decomposition [Ziletti and D’Ambrosi, 2025].

Example ontology.

Table 4: Example axes in the clinical facet ontology.

Axis	Values
Primary condition	encephalitis/myelitis, heart failure, pneumonia, type 2 diabetes, chronic kidney disease, sepsis, stroke, COPD, liver cirrhosis, pulmonary embolism
Demographics	age < 40, age 40–64, age 65–75, age > 75, male, female, race/ethnicity group if ethically justified and handled carefully
Comorbidities	uncomplicated diabetes, complicated diabetes, renal disease, congestive heart failure, chronic pulmonary disease, dementia, malignancy, liver disease, immunosuppression
Interventions	antibiotics, antivirals, steroids, anticoagulation, dialysis, oxygen escalation, rehabilitation, surgery, medication adjustment
Durations	short course, prolonged course, delayed initiation, early transition, recurrent treatment
Outcomes	home discharge, skilled nursing facility, inpatient rehab, readmission, functional improvement, persistent deficit, complication
Reasoning relation	comparison, contrast, causal relation, temporal trigger, subgroup difference, management trade-off

Facet template. Each facet should be represented as a structured object:

```
{
  "facet_id": "F_encephalitis_elderly_rehab_outcome",
  "primary_condition": "encephalitis",
  "subgroup": {"age": ">75"},
  "clinical_axis": "rehabilitative_outcome",
  "answer_role": "comparison_arm",
  "minimum_gold_chunks": 6,
  "target_cluster_size": 15
}
```

Important constraint. Facets should be individually answerable but collectively necessary. A single chunk must not answer the whole query. This is what creates the multi-source retrieval problem.

Facets should also be objectively labelable from the hidden schema. Prefer labels such as age group, comorbidity presence, treatment-duration category, discharge disposition, complication type, rehabilitation setting, or explicitly stated response to therapy. Avoid facets that require ambiguous clinical judgment unless they are backed by an explicit rubric, because subjective labels make retrieval evaluation hard to interpret [Li et al., 2023].

7.2 Stage 2: Hidden Query Plans

Purpose. Generate query plans from the catalogue before writing query text or evidence chunks. A query plan is the hidden recipe that says which facets must be covered, which clusters are gold, which cluster is intentionally dominant, which distractor families should be generated, and what the answer must contain. This follows the answer-first idea in MHTS [Lee et al., 2025], the query-answer-clue mapping in DRAGON [Shen et al., 2026], the question-aware construction of EnterpriseRAG-Bench [Sun et al., 2026], and the hidden symbolic plan used by PhantomWiki [Gong et al., 2025].

Blueprint structure.

```
{
  "query_id": "Q_000142",
  "primary_condition": "encephalitis_or_myelitis",
  "query_type": "subgroup_comparison",
  "facets": [
    "F_diabetes_treatment_duration",
    "F_diabetes_rehab_outcome",
    "F_elderly_treatment_duration",
    "F_elderly_rehab_outcome"
  ],
  "contrast_relation": "compare diabetes vs age_over_75",
  "template_id": "subgroup_comparison_two_axes",
  "logical_form": {
    "condition": "encephalitis_or_myelitis",
    "arms": ["uncomplicated_diabetes", "age_over_75"],
    "axes": ["treatment_duration", "rehab_outcome"],
    "operator": "compare"
  },
  "expected_answer_outline": [
    "describe treatment duration in uncomplicated diabetes subgroup",
    "describe rehabilitative outcomes in uncomplicated diabetes subgroup",
    "describe treatment duration in age_over_75 subgroup",
    "describe rehabilitative outcomes in age_over_75 subgroup",
    "synthesize differences and caveats"
  ]
}
```



```

],
"difficulty": {
  "n_facets": 4,
  "dominant_gold_cluster": "C1",
  "dominant_gold_chunks": 18,
  "complementary_gold_chunks_per_cluster": 8,
  "hard_distractor_chunks": 24,
  "expected_topk_failure": "oversample_dominant_gold_cluster"
},
"split": "validation"
}

```

The output of this stage is `query_plans.parquet`. The important point is that diversity is already specified here: the plan says which clusters are gold, which one is intentionally redundant, which facets are complementary, and which distractor families should be generated.

Template and logical-form layer. PhantomWiki uses a context-free grammar and a Prolog solver to generate questions with verifiable answers [Gong et al., 2025]. The medical benchmark does not need to expose Prolog to the retrieval system, but it should copy the separation of concerns:

- **template_id**: names the question family, such as subgroup comparison, constrained cohort, conflicting evidence, completeness, or outcome synthesis;
- **logical_form**: stores the condition, subgroups, axes, constraints, and comparison/aggregation operator;
- **answer_facts**: lists the atomic answer facts expected from each facet;
- **anti_hallucination_facts**: stores false or distractor-derived claims that should not appear in the answer.

This makes the LLM responsible for natural language rendering, not for deciding the hidden truth of the item.

Difficulty control. Assign a difficulty score:

$$D(q) = \alpha \cdot n_{\text{facets}} + \beta \cdot n_{\text{clusters}} + \gamma \cdot \bar{d}_{\text{facet}} + \delta \cdot n_{\text{distractor types}} + \eta \cdot n_{\text{reasoning steps}} + \zeta \cdot \log |V|,$$

where $\bar{d}_{\text{facet}}$ is the average semantic distance among gold facet clusters after embedding and $|V|$ is the candidate or corpus size used for the retrieval setting. This adapts MHTS’s point that difficulty is not just hop count [Lee et al., 2025] and PhantomWiki’s separation between reasoning difficulty and retrieval difficulty [Gong et al., 2025].

7.3 Stage 3: Generate Structured Fact Rows

Purpose. Create the hidden structured truth from which chunks and answers are generated. Each fact row says what the generated text is allowed to mention and which facet it supports. This stage follows MedDialogRubrics’ principle of grounding patient simulation in atomic medical facts [Gong et al., 2026] and PhantomWiki’s principle that documents and answers should be derived from a consistent hidden world [Gong et al., 2025].

Atomic fact example.

```

{
  "fact_id": "FACT_003819",
  "patient_profile_id": "P_10482",

```

```

"encounter_id": "E_22194",
"primary_condition": "encephalitis",
"concept_ids": ["C0014038", "C0009450"],
"age": 79,
"age_group": ">75",
"comorbidities": ["hypertension", "atrial_fibrillation"],
"intervention": "acyclovir",
"treatment_duration": "14 days",
"rehab_disposition": "inpatient rehabilitation",
"outcome": "persistent gait instability but improved orientation",
"facet_ids": [
  "F_elderly_treatment_duration",
  "F_elderly_rehab_outcome"
],
"must_mention": [
  "14-day antiviral course",
  "age over 75",
  "inpatient rehabilitation",
  "gait instability"
],
"must_not_mention": [
  "diabetes",
  "dialysis",
  "complete recovery"
]
}

```

For each facet, generate enough rows to create a cluster. These rows should vary across synthetic patients while preserving the same facet meaning. A structured fact row may contain fields useful for more than one answer claim, but rendered chunks should usually be scoped to one facet unless the benchmark intentionally includes an easy ablation. The output of this stage is `clinical_facts.parquet`.

Structured rule-check layer. For the MVP, implement this as pandas filters over `clinical_facts.parquet`. Do not introduce SQL, Datalog, or Prolog until the dataframe version works. The goal is not to encode all medical reasoning. The goal is to make benchmark labels mechanically checkable:

- a facet query returns the exact fact rows that support that facet;
- a distractor query returns rows that match some surface terms but fail at least one required constraint;
- a completeness query returns all required rows, not just one representative;
- a conflicting-evidence query identifies the newer or higher-priority row and the superseded row.

This is the clinical analogue of PhantomWiki’s solver-derived exhaustive answers [Gong et al., 2025], but implemented with ordinary dataframe logic. It reduces the need to ask an LLM to infer qrels after generation.

Generation method. Use attribute-guided generation [Huang et al., 2025]. The prompt should specify:

- the primary condition;
- normalized concept IDs or relation triples when available;
- the demographic subgroup;

- comorbidities;
- allowed interventions;
- target outcome;
- the exact facet IDs the fact should support;
- forbidden facts to prevent leakage across facets.

Validation. Reject a generated fact if:

- required fields are missing;
- the clinical relation is internally inconsistent;
- it mentions a forbidden subgroup;
- it supports more facets than intended;
- it contains an impossible treatment/outcome pair.

7.4 Stage 4: Render Fact Rows into Text Chunks via LLM

Purpose. Render structured facts into realistic text chunks that can be embedded and retrieved.

Template-first, LLM-second rendering. PhantomWiki deliberately uses templates because unconstrained LLM rephrasing can introduce factual errors [Gong et al., 2025]. This benchmark needs more natural clinical prose than PhantomWiki’s minimal articles, but the same warning applies. Use a two-pass rendering policy: first create a deterministic or tightly templated draft from the fact row, then let the LLM rewrite it under strict `must_mention/must_not_mention` constraints and re-extract the structured facts afterward. Reject rewrites whose extracted facts differ from the source row.

Chunk styles. Generate multiple styles so the corpus is not one-note:

- discharge-summary brief hospital course;
- progress-note assessment and plan;
- rehab consultation summary;
- problem-list paragraph;
- outcome follow-up note;
- medication-management note.

Narrative generation prompt.

You are generating a synthetic clinical evidence chunk for a RAG benchmark.
Use only the structured facts provided below.
Do not add diagnoses, treatments, durations, outcomes, or demographics that are not present in the facts.

Style: brief hospital course problem-list paragraph.
Length: 90-160 words.
Required facet IDs: {facet_ids}
Required facts: {atomic_facts}
Forbidden facts: {forbidden_facts}

Write one self-contained clinical paragraph. It should sound like a deidentified clinical note, but it must not include names, dates, addresses, or real identifiers.

Traceability requirement. Every chunk must retain:

- `fact_ids`: which facts were used;
- `facet_ids`: which facets the chunk supports;
- `query_id`, `cluster_id`, `cluster_role`, and `split`;
- `span_offsets`: optional character offsets for fact mentions;
- `generation_prompt_id`: which prompt generated it.

The output of this stage is `chunks.parquet`.

Why not rely on LLM annotation afterward? Because the old pipeline used LLM map-reduce over candidate chunks to discover gold facets after retrieval. That makes gold annotation dependent on noisy text and candidate-pool choices. Here, the gold labels are known before embedding. LLMs can audit the labels, but they should not be the only source of truth.

7.5 Stage 5: Create Role-Aware Gold Clusters

Purpose. Facility-location wins when there are dense redundant regions and multiple complementary regions. Therefore, each facet must have a cluster of several relevant chunks.

Cluster design. For each query facet:

- generate 5–20 chunks that support that facet;
- vary patients, wording, and note style;
- keep the same facet-level answer;
- avoid generating exact paraphrases only;
- preserve planned within-cluster redundancy while removing accidental cross-facet duplication;
- ensure chunks within a facet are closer to each other than to chunks in other facets.

Role-aware deduplication. Duplicate filtering must not erase the experimental condition. Exact copies and accidental near-duplicates across facets should be rejected, but planned redundancy inside a dominant or complementary cluster should be preserved up to its target size. Otherwise the pipeline removes the very top- k failure mode it is trying to test.

Completeness clusters. EnterpriseRAG-Bench creates small groups of mutually visible documents for completeness questions, with facts deliberately distributed so that no single document contains the full answer [Sun et al., 2026]. The clinical version should create optional `completeness_cluster` groups: 4–10 chunks tied to the same answer facet family, where every chunk contributes a necessary subclaim. These clusters are useful for testing whether facility-location retrieves one representative per facet and whether increasing k improves answer-fact completeness rather than only retrieving redundant notes.

Example. For the query:

For patients diagnosed with encephalitis or myelitis, how do treatment durations and rehabilitative outcomes differ between those with uncomplicated diabetes and elderly patients over age 75?

generate these gold clusters:

1. encephalitis + uncomplicated diabetes + treatment duration;
2. encephalitis + uncomplicated diabetes + rehab outcome;
3. encephalitis + age > 75 + treatment duration;
4. encephalitis + age > 75 + rehab outcome.

Top- k should be tempted to over-sample the most query-similar cluster, for example encephalitis + diabetes + treatment, because it shares many exact query terms. Facility-location should spend fewer picks in that redundant cluster and allocate representatives to the other clusters.

7.6 Stage 6: Generate Hard Distractor Chunks

Purpose. The benchmark must distinguish coverage from mere dispersion. MMR should not automatically win by selecting semantically distant chunks. Therefore, add distractors that are diverse but not relevant.

Distractor types.

Table 5: Distractor categories and expected retrieval behavior.

Distractor type	Example	Expected effect
Same condition, wrong subgroup	encephalitis in middle-aged patients without diabetes	Top- k may retrieve these because condition terms match.
Same subgroup, wrong condition	elderly patients with pneumonia rehab outcomes	MMR may retrieve these because they add demographic/outcome diversity.
Same intervention, wrong disease	acyclovir duration for shingles rather than encephalitis	Similar treatment vocabulary creates hard negatives.
Same outcome, wrong clinical axis	rehab outcome after stroke, not encephalitis	Outcome terms match but facet is wrong.
Broad textbook-style background	general encephalitis treatment principles	Tests whether the query is answerable only from corpus-specific evidence.
Rare outlier	unusual complication unrelated to the query	MMR may overvalue outliers; facility-location should ignore isolated points.

Important distinction. The distractors should be semantically plausible, not nonsense. A trivial distractor does not test retrieval quality.

Noise and contradiction policy. EnterpriseRAG-Bench adds realistic noise through misfiled documents, near-duplicates with controlled factual changes, miscellaneous files, and conflicting information [Sun et al., 2026]. The clinical benchmark should adapt these as follows:

- **Misfiled/source-style noise:** a rehabilitation-relevant fact rendered in a discharge-summary style or a discharge-disposition fact rendered as a follow-up note, with labels preserved;
- **Near-duplicate clinical noise:** a chunk copied in structure but with one controlled change, such as 10-day vs. 14-day treatment duration;
- **Conflicting-evidence pairs:** older vs. newer synthetic notes where the query asks for the most recent or reconciled pattern;

- **Miscellaneous/background notes:** loosely related general clinical summaries that share vocabulary but do not satisfy the query-local constraints.

All such noise must be generated intentionally and tracked in metadata. Otherwise noise becomes uncontrolled label ambiguity rather than a retrieval stressor.

For the MVP, implement only the first three hard distractor types in Table 3. Leave misfiled/source-style noise, near-duplicate contradictions, and information-not-found questions until the core facet-coverage experiment works.

7.7 Stage 7: Generate the Natural Query and Gold Answer via LLMs

Purpose. Generate realistic broad clinical queries while preserving the hidden facet decomposition.

This stage draws from MQ4CS/MASQ’s multi-aspect query framing [Abbasiantaeb et al., 2026], CliniQG4QA’s question diversity idea [Yue et al., 2021], ScIRGen’s use of cognitive/question taxonomies [Lin et al., 2025], EnterpriseRAG-Bench’s question families [Sun et al., 2026], and PhantomWiki’s grammar-controlled question generation [Gong et al., 2025].

Query types.

- subgroup comparison: “How do outcomes differ between A and B?”
- management trade-off: “How is treatment adjusted when condition X co-occurs with Y?”
- temporal trigger: “Which findings tend to trigger escalation from treatment A to B?”
- outcome synthesis: “What discharge and rehabilitation patterns are reported across subgroups?”
- complication contrast: “Which complications change the management pathway?”
- constrained cohort: “Among patients matching A and B, which treatment pattern appears under constraint C?”
- conflicting evidence: “How should the treatment duration be interpreted when earlier and later notes disagree?”
- completeness: “Across all documented subgroups, which outcomes are reported and which subgroup does each belong to?”
- information not found: “Does the corpus report X for subgroup Y?” where the correct answer is absence.

Template-controlled generation. Use a finite set of query templates before asking an LLM to write fluent text. This is a softer clinical version of PhantomWiki’s context-free grammar [Gong et al., 2025]: the template fixes the required arms, axes, constraints, aggregation operator, and expected number of answer facts, while the LLM only produces natural wording. This prevents the generator from silently dropping one facet or adding a new one.

Query generation prompt.

```
Generate one broad clinical research-style question for a RAG benchmark.
The question must require evidence from all listed facets.
It must not reveal the facet IDs.
It must not be answerable from general medical knowledge alone; it should ask
about patterns in the provided synthetic corpus.

Primary condition: {condition}
Required facets:
{facet_descriptions}
Question type: {query_type}
```

```

Target difficulty: {difficulty}

Return JSON:
{
  "query_text": "...",
  "subqueries": ["...", "...", "..."],
  "why_multi_aspect": "...",
  "forbidden_single_source_answer": true
}

```

Subqueries. Store subqueries but do not use them for the main top- k /MMR/facility-location comparison unless running a separate multi-query retrieval ablation. The main experiment should rank from the single natural query so that methods face the same information need.

Gold answer construction. The answer is also produced at this stage, during benchmark construction, before embedding and before any retrieval method is evaluated. The source of truth is the hidden scaffold: `query_plans.parquet`, `clinical_facts.parquet`, and the fact-to-chunk mapping stored in `chunks.parquet`. This is close to DRAGON’s explicit query–answer–clue–source mapping [Shen et al., 2026].

Use two separate prompts or procedures:

1. **Query prompt:** given only the hidden plan, write the user-facing `query_text`; do not expose the answer wording.
2. **Answer prompt:** given the required facets, the answer outline, and the structured fact rows grouped by facet, write a canonical answer. The prompt may receive `fact_ids` and their derived `chunk_ids` for citation, but it should not receive distractor chunks or retrieval rankings.

Answer format.

```

{
  "query_id": "Q_000142",
  "answer_text": "Across the synthetic corpus, patients with ...",
  "facet_answer_summaries": [
    {
      "facet_id": "F_diabetes_treatment_duration",
      "summary": "...",
      "required_chunk_ids": ["C_001", "C_017"]
    }
  ],
  "minimum_sufficient_evidence": {
    "one_chunk_per_facet": true,
    "required_facets": ["F1", "F2", "F3", "F4"]
  }
}

```

Avoiding leakage. The answer should not introduce new facts not present in `clinical_facts.parquet`. Use an LLM judge only to check consistency:

- every answer claim is supported by at least one fact;
- every required facet is mentioned;
- no unsupported clinical recommendation is added;
- no distractor fact appears in the answer.

For constrained, conflicting, and hard-distractor query families, also store `anti_hallucination_facts`. EnterpriseRAG-Bench uses these negative facts to catch systems that answer from a plausible distractor rather than the intended evidence [Sun et al., 2026]. In this benchmark they can be used during answer evaluation and validation, not during retrieval.

The output of this stage is `queries.parquet` and `gold_answers.parquet`.

7.8 Stage 8: Create Qrels Directly from the Hidden Mapping

Purpose. Create `qrels.parquet` from the known mapping between queries, chunks, facts, and facets. The LLM-generated text is not re-annotated after retrieval to discover gold labels; qrels are derived from the hidden construction graph.

Example qrels.

query_id	chunk_id	facet_id	relevance_grade	fact_ids
Q_001	CH_017	F3	1	[FACT_001]
Q_001	CH_017	F4	1	[FACT_001]
Q_001	CH_088	none	0	[]

If a chunk is allowed to support multiple facets, every supported query–chunk–facet relationship must be represented explicitly. Distractor chunks receive no positive gold qrel for the target query, even when they are semantically close.

7.9 Stage 9: Validate the Generated Objects

Validation agents. Use a multi-agent curation pattern similar to DATAGEN and Driouich et al. [Huang et al., 2025, Driouich et al., 2025]:

1. **Schema validator:** checks JSON fields and allowed values.
2. **Clinical plausibility validator:** checks that the case is medically plausible.
3. **Facet-support validator:** checks whether the chunk supports exactly the intended facet(s).
4. **Diversity validator:** computes similarity, checks cluster quality, and applies role-aware duplicate handling.
5. **No-leakage validator:** checks that distractors do not accidentally answer a gold facet.
6. **Non-copying validator:** checks that MIMIC-seeded style examples or guideline snippets were not copied into the synthetic corpus.
7. **Re-extraction validator:** extracts schema fields back from generated chunks and compares them with the hidden facts.
8. **Profile and subgroup validator:** checks facet balance, profile-level split integrity, and obvious protected-attribute artifacts.
9. **No-context baseline validator:** checks whether a model can answer the query without retrieval.
10. **Multi-retriever pooling validator:** runs BM25, vector retrieval, and optionally an agentic/file-system retriever, then checks whether pooled retrieved chunks expose accidental gold evidence, missing qrels, or stronger distractors than intended.

The pooling validator is adapted from EnterpriseRAG-Bench [Sun et al., 2026]. In this thesis benchmark, pooled corrections should normally cause regeneration or rejection rather than silent gold-set mutation, because the hidden schema is meant to define the ground truth. Still, pooling is valuable because it finds collisions at corpus scale that a query-local generator may miss.

Automatic checks. Run these checks before embedding:

Table 6: Recommended validation checks.

Check	Metric	Default action
Structured completeness	all required fields non-null	reject object
Query facet count	every query has 3–5 required facets	regenerate query plan
Facet balance	gold chunks per facet in target range	regenerate underrepresented facets
Cluster role consistency	every <code>cluster_id</code> has exactly one <code>cluster_role</code>	reject or repair cluster meta-data
Traceability	every chunk traces back to fact IDs and facet IDs	reject chunk
Forbidden fact leakage	generated chunk mentions a <code>must_not_mention</code> fact	rewrite or reject chunk
Near-duplicate chunks	cosine similarity above threshold, e.g. > 0.94	role-aware handling: preserve planned redundancy, reject exact copies and accidental cross-facet duplicates
Authenticity/non-copying	overlap against any MIMIC seed or guideline source above threshold	rewrite or reject object
Cluster quality	intra-facet similarity, inter-facet separation, silhouette or related score	regenerate weak clusters
Distractor leakage	distractor judged relevant to required facet	relabel or reject
Query answerability	every facet has at least one gold chunk	reject query
Anti-hallucination facts	distractor-derived false claims are not supported by gold chunks	repair distractor metadata or reject query
Multi-retriever collision check	non-gold pooled chunks judged required or valid for the query	add explicit qrel if schema agrees, otherwise regenerate or reject
Single-chunk shortcut	one chunk supports all required facets	reject or downgrade difficulty
Canonical answer coverage	answer uses every required facet at least once	rewrite answer or reject query
No-retrieval answerability	LLM without context answers too well	reject query or make it more corpus-specific

Why the no-context check matters. RAGVAL emphasizes that public or generic knowledge can make a RAG benchmark misleading because the generator may answer without retrieval [Kenneweg et al., 2024]. In this benchmark, queries should ask about corpus-specific synthetic patterns, not universal facts such as “What is encephalitis?”.

7.10 Stage 10: Embed and Filter by Retrieval Geometry

Purpose. The dataset should not be accepted merely because the hidden labels look good. After embedding, the candidate pools must exhibit the intended geometry. These are benchmark-construction checks, so they may use hidden fields such as `query_id`, `facet_ids`, `cluster_id`, `cluster_role`, `distractor_type`, and `qrels.parquet`. These fields validate the dataset; they are not exposed to retrieval methods.

Procedure. Embed all chunks and queries. Then separate two different questions:

- **Query-local structure, primary generation check:** use only the chunks generated for the current query. This checks whether the synthetic item itself has the intended facet and cluster structure.
- **Full-corpus visibility, primary retrieval check if full-corpus evaluation is reported:** use the full `chunks.parquet` corpus. This checks whether the query’s gold facets remain visible in the first-stage top- N pool when chunks from other query plans also compete.

Query-local checks.

- each required facet has enough gold chunks;
- intra-facet similarity is high enough that each facet forms a usable cluster;
- inter-facet similarity is lower than intra-facet similarity, so facets do not collapse into one cluster;
- the dominant gold cluster is larger or more query-similar than the complementary gold clusters;
- hard distractors are close to the query or to a gold facet, but have no gold qrel for the target query.

Corpus-level geometry diagnostics. EnterpriseRAG-Bench validates synthetic corpus structure by comparing local nearest-neighbor density, in-cluster similarity, and cross-cluster similarity across corpora [Sun et al., 2026]. Use the same family of diagnostics here, but compute them query-locally and full-corpus:

- **Local density:** average cosine similarity from each gold or hard-distractor chunk to its top-10 nearest neighbors;
- **In-facet similarity:** average pairwise cosine similarity among chunks in the same gold facet cluster;
- **Cross-facet similarity:** average pairwise cosine similarity across different gold facet clusters;
- **Distractor proximity:** average similarity from hard distractors to the query and to their nearest gold facet cluster;
- **Scaling curve:** repeat retrieval at several corpus sizes to check that first-stage recall degrades smoothly rather than collapsing due to accidental corpus collisions.

The desired structure is not maximum separation. The task is useful when gold facets form coherent clusters, complementary facets remain visible but distinct, and hard distractors lie close enough to challenge top- k and MMR without becoming relevant.

Full-corpus visibility checks. If the main evaluation uses full-corpus retrieval:

- each required facet has at least one gold chunk in the full-corpus top- N pool;
- the top- N pool contains hard distractors or background negatives, so evaluation is not trivial;
- chunks from other query plans do not completely swamp the query’s gold evidence.

Candidate query filters.

$$\text{FacetPresence}(q) = \frac{|\{f : \exists c \in P_N(q), c \in \text{Gold}(q, f)\}|}{|\text{Facets}(q)|}, \quad (1)$$

$$\text{TopKDominance}(q, k) = \max_f \frac{|\{c \in \text{TopK}(q, k) : c \in \text{Cluster}(f)\}|}{k}, \quad (2)$$

$$\text{GoldPoolRecall}(q, N) = \frac{|\text{Gold}(q) \cap P_N(q)|}{|\text{Gold}(q)|}, \quad (3)$$

$$\Delta_{\text{cluster}}(q) = \bar{s}_{\text{in-facet}} - \bar{s}_{\text{cross-facet}}. \quad (4)$$

Recommended keep rules are structural admission filters, fixed before final testing:

- $\text{FacetPresence}(q) = 1.0$ for the chosen N ;
- $\text{TopKDominance}(q, 10) \geq 0.5$ for at least one common k ;
- $\text{GoldPoolRecall}(q, N) \geq 0.5$ or, better, use pool-relative qrels;
- $\Delta_{\text{cluster}}(q)$ is positive and above a predeclared threshold;
- hard distractors are close enough to appear in the candidate pool without receiving positive qrels;
- query-local facet clusters remain separated rather than collapsing into one broad cluster.

Important warning. Do not evaluate against gold chunks that are structurally absent from the candidate pool. Either ensure all gold facets have representatives in the pool or compute recall against *pool-visible gold*. The old MIMIC setup likely suffered from annotation/evaluation mismatch: the annotation pool and full-corpus evaluation pool were not identical.

7.11 Stage 11: Evaluate Retrieval Methods

Corpus settings. The retrieval corpus is part of the benchmark definition, so different corpus choices should be reported as different settings:

- **Full-corpus retrieval, primary setting:** use the full generated corpus for the split. The corpus contains gold chunks for the current query, its hard distractors, and chunks generated for other query plans.
- **Query-local retrieval, diagnostic ablation:** run the same methods on only the chunks generated for the current query: gold chunks, hard distractors, and query-specific background negatives. This tests reranking behavior after the query-local universe has already been isolated.

Retrieval methods.

1. **Top- k :** select the k highest cosine-similarity chunks.
2. **MMR:** greedily select chunks balancing query similarity and distance from selected chunks.
3. **Facility-location:** greedily optimize a coverage objective over the candidate pool.
4. **BM25 and hybrid retrieval, optional sanity baselines:** include at least one lexical baseline if the final benchmark claims realistic RAG behavior, because EnterpriseRAG-Bench shows that vector retrieval may underperform lexical retrieval in domains with specialized terms and identifiers [Sun et al., 2026].

Facility-location objective. Let V be the candidate pool, $S \subseteq V$ be the selected set, s_{ij} be chunk–chunk similarity, r_i be query relevance, and $w_i = g(r_i)$ be a nonnegative query-relevance weight, for example a clipped or normalized score derived from first-stage similarity. A useful objective is:

$$F(S) = \lambda \sum_{i \in V} w_i \max_{j \in S} s_{ij} + (1 - \lambda) \sum_{j \in S} r_j.$$

The weighted coverage term rewards representing high-relevance regions of the candidate pool rather than every dense region equally; the relevance term prevents the method from selecting low-relevance representatives. This matters because the benchmark intentionally contains dense distractor clusters. Tune λ and any weighting function parameters on the validation split only.

MMR objective.

$$\text{MMR}(c) = \lambda \text{sim}(q, c) - (1 - \lambda) \max_{s \in S} \text{sim}(c, s).$$

MMR promotes novelty relative to already selected chunks. It can help, but it may select isolated outliers or semantically diverse distractors. The proposed benchmark should make this difference visible: coverage should prefer dense, relevant facet clusters; MMR may overvalue mere distance.

8 Metrics

The most important metric is not ordinary document recall. The benchmark is designed around facets.

Table 7: Recommended evaluation metrics.

Metric	Definition and purpose
Facet Coverage@k	Fraction of required facets represented by at least one retrieved gold chunk. In this pipeline each required facet is implemented as one gold cluster, so this is also cluster coverage. Primary metric.
Weighted Facet Coverage@k	Average fraction of pool-visible gold chunks retrieved per facet. Captures depth of coverage.
Gold Precision (GP)	Fraction of retrieved chunks that are gold for any facet. Guards against retrieving many irrelevant representatives.
Gold Recall (GR)	Fraction of pool-visible gold chunks retrieved. Report pool-visible recall to avoid penalizing methods for absent gold chunks.
Dominant-Cluster Concentration@k	Fraction of selected chunks drawn from the largest or intentionally dominant gold cluster. Useful for showing the top- k redundancy failure mode.
Redundancy@k	Average number of selected chunks per represented facet or mean pairwise similarity among selected gold chunks. Lower is better if facet coverage is preserved.
Distractor Rate@k	Fraction of selected chunks that are hard distractors. Useful for showing MMR failure modes.
Facet nDCG	Graded ranking metric where each facet contributes gain once or with diminishing returns.
Answer Fact Completeness	Fraction of canonical answer facts whose supporting facet has been retrieved. This adapts EnterpriseRAG-Bench’s atomic answer-fact completeness metric to the pre-generation clinical fact table.

Metric	Definition and purpose
Invalid Extra Chunks	Absolute count of selected chunks that are neither gold nor explicitly valid background. This captures how much irrelevant context a downstream answer generator must process.
Answer Citation Coverage	Whether a downstream answer generator has enough retrieved citations to support every answer facet.
Facility-location Score	Objective value, reported as a diagnostic, not as the main external metric.
Jaccard vs. Top- k	Measures how much a method actually differs from the baseline.

Primary thesis table. The main result should report Facet Coverage@ k , Weighted Facet Coverage@ k , GP, GR, Dominant-Cluster Concentration@ k , and Distractor Rate@ k for each method at $k \in \{5, 10, 20, 30\}$.

Expected result pattern. The desired pattern is:

- facility-location has the highest facet coverage;
- top- k has high precision in the dominant facet but lower facet coverage;
- MMR has higher diversity than top- k but worse GP and higher distractor rate than facility-location;
- the gap is largest at small or moderate k , e.g. $k = 5$ and $k = 10$;
- at high k , top- k may catch up because it eventually retrieves many clusters by volume.

9 Concrete Generation Example

9.1 Hidden Query Plan

```

Primary condition:
  encephalitis or myelitis

Required comparison groups:
  A: uncomplicated diabetes
  B: age over 75

Required clinical axes:
  1. treatment duration
  2. rehabilitative outcome

Gold facets:
  F1: diabetes + treatment duration
  F2: diabetes + rehabilitative outcome
  F3: elderly + treatment duration
  F4: elderly + rehabilitative outcome

```

9.2 Generated Natural Query

For patients diagnosed with encephalitis or myelitis, how do treatment durations and rehabilitative outcomes differ between those with uncomplicated diabetes and elderly patients over age 75?

9.3 Gold Cluster Plan

Table 8: Example gold and distractor clusters for one query.

Cluster	Contents	Gold role	Target size
C1	encephalitis + uncomplicated diabetes + antiviral duration	dominant gold, facet F1	18 chunks
C2	encephalitis + uncomplicated diabetes + rehab outcome	complementary gold, facet F2	8 chunks
C3	encephalitis + age > 75 + treatment duration	complementary gold, facet F3	8 chunks
C4	encephalitis + age > 75 + rehab outcome	complementary gold, facet F4	8 chunks
D1	encephalitis + no diabetes + age 40–64	hard distractor	20 chunks
D2	elderly + stroke + rehab outcome	hard distractor	20 chunks
D3	diabetes + pneumonia + treatment duration	hard distractor	20 chunks
D4	general encephalitis background	generic distractor	5 chunks

9.4 Expected Retrieval Behavior

Top- k should retrieve many C1 chunks because the natural query contains condition, diabetes, and treatment-duration terms. MMR should retrieve from C1 but then jump toward semantically distinct clusters, including D2 or D3. Facility-location should select representatives from C1–C4 because they are dense regions in the candidate pool and each contributes coverage.

10 Recommended Ablations

1. **No distractors:** verifies that all methods improve when the task is easy.
2. **No redundancy:** removes the condition under which facility-location should help; expected to shrink the gap.
3. **No structured fact grounding:** use naive LLM-generated notes and show quality/gold-label degradation.
4. **MIMIC-seeded vs. fully synthetic:** compare style realism and retrieval geometry.
5. **Different embedding models:** general embedding vs. biomedical embedding.
6. **Single-query vs. subquery fusion:** tests whether explicit multi-aspect query decomposition competes with reranking.
7. **Facet count:** 2, 3, 4, and 5 facets per query.
8. **Distractor strength:** easy, medium, hard, and outlier distractors.

11 Prompt and Model Strategy

Generator model. Do not assume that the best answering model is the best generator. AGORABench shows that model generation ability and task-solving ability can diverge [Kim et al., 2025]. If resources permit, compare two generator models on a small pilot batch before freezing the generation pipeline, then choose the one with better structured validity, diversity, and clinical plausibility.

Temperatures. Use low temperature for structured facts and validation. Use moderate temperature for narrative variation. DATAGEN reports diversity benefits from generation settings and attribute guidance [Huang et al., 2025]; however, benchmark labels should remain deterministic.

Prompt optimization. If generation quality is unstable, an optional later phase can optimize prompts for two objectives: clinical fidelity and diversity. This is inspired by multi-objective prompt learning [Pastrián et al., 2026], but it is not necessary for the first implementation.

12 Implementation Steps

1. Create `ontology.yaml` with conditions, concepts, comorbidities, demographics, interventions, outcomes, and allowed combinations.
2. Create `archive_scaffold.yaml` with condition-family overviews, care-path templates, note/source styles, and style-agent instructions.
3. Create `profile_templates.yaml` or an equivalent latent-profile table for schema-first generation and split control.
4. Generate `query_plans.parquet`: sample query family + condition + 3–5 objective facets + difficulty label + optional template/logical form.
5. Generate `clinical_facts.parquet`: structured atomic facts for each facet and distractor.
6. Run structured solver/rule checks to derive expected answer facts, anti-hallucination facts, and exhaustive facet memberships.
7. Validate facts using schema, clinical plausibility, concept-consistency, and subgroup-balance checks.
8. Generate `chunks.parquet`: narrative text from facts.
9. Apply role-aware deduplication, non-copying checks, and group-balance enforcement.
10. Generate `queries.parquet`: natural query text plus hidden subqueries and optional surface variants.
11. Generate `qrels.parquet`: directly from `chunk_id-fact_id-facet_id` mappings.
12. Generate `gold_answers.parquet`: one answer per query with facet citations.
13. Run multi-retriever pooling to detect accidental gold chunks, hidden collisions, or overpowered distractors.
14. Write `generation_audit.parquet`: prompt hashes, model versions, decoding parameters, seeds, validators, rejection reasons.
15. Freeze validation/test splits by query family/profile before final threshold tuning.
16. Embed chunks and queries.
17. Compute candidate pools and geometry diagnostics.

18. Filter queries based on predeclared facet presence, top- k dominance, pool-visible gold, distractor proximity, and query-local cluster separation.
19. Tune λ on validation.
20. Report final metrics on test.

13 Quality Report to Include in the Thesis

The benchmark should include a transparent generation report. Following the concerns raised in biomedical synthetic data surveys [Rao et al., 2026, Montenegro et al., 2025], report:

- number of generated facts, chunks, queries, facets, and distractors;
- rejection rates by validation stage;
- average chunks per facet;
- distribution of query types and difficulty levels;
- average within-facet and across-facet embedding similarity;
- local nearest-neighbor density and distractor-proximity diagnostics;
- retrieval scaling curves across several corpus or candidate-pool sizes;
- no-context answerability rate;
- prompt hashes, model identifiers, decoding settings, schema versions, and random seeds;
- profile-level and subgroup-level split distributions;
- non-copying/authenticity check results when real examples seed the generator;
- cluster-quality diagnostics such as intra-facet similarity, inter-facet separation, and silhouette scores;
- human audit size and agreement if available;
- examples of accepted and rejected generations;
- limitations of synthetic clinical realism.

14 Threats to Validity

Synthetic bias. The corpus is generated to favor coverage. This is acceptable if the thesis frames the benchmark as a controlled stress test. It would be problematic only if presented as evidence that facility-location always improves real clinical RAG.

Authenticity and privacy. If MIMIC-IV snippets or guideline text are used as style seeds, generated chunks may accidentally copy source phrasing. Mitigate with abstraction prompts, overlap checks, source hashes, and manual spot audits. The final corpus should contain synthetic evidence, not republished patient text.

Medical realism. LLM-generated facts may be clinically imperfect. Mitigate with constrained templates, concept grounding, decision-pathway grounding, and validation. Avoid using the dataset for clinical recommendations.

Embedding dependence. The result may depend on the embedding model. Report experiments with at least one general-purpose and one biomedical embedding model if feasible.

Query filtering bias. Filtering for geometric properties can look like cherry-picking. Mitigate by defining filters before final evaluation, applying them uniformly, and reporting how many queries were rejected for each reason.

Gold-label leakage. If narrative chunks accidentally mention all facets, the task becomes single-source. Reject chunks that support too many facets unless intentionally creating an easy split.

Generated-world overfitting. If only one static synthetic corpus is used, systems and prompt choices may overfit to that instance. Mitigate by reporting seeds, generating at least one fresh held-out instance where feasible, and separating validation from test at the query-family/profile level. This follows PhantomWiki’s on-demand benchmark motivation [Gong et al., 2025].

Correction-aware evaluation tension. EnterpriseRAG-Bench treats gold sets as revisable hypotheses at large scale [Sun et al., 2026]. This clinical benchmark should be stricter because labels are generated from a hidden schema, but multi-retriever pooling may still reveal collisions. Report how often pooled evidence caused regeneration, qrel repair, or query rejection.

MMR parameter fairness. MMR must be tuned on the same validation split as facility-location. Otherwise, the comparison is not fair.

15 Final Recommended Experimental Claim

The strongest thesis claim is:

We construct a controlled synthetic clinical RAG benchmark in which each query is generated from an explicit multi-facet evidence plan. The corpus contains redundant relevant clusters, complementary gold clusters, and hard distractors. Under this controlled multi-aspect setting, facility-location reranking improves facet-level evidence coverage over similarity top- k and reduces the distractor sensitivity of dispersion-only MMR.

This claim is narrower than “facility-location is better for medical RAG”, but it is much more defensible and experimentally clean.

16 Minimal First Version

If time is limited, implement the smallest useful version:

- 4 primary conditions;
- 100 queries;
- 4 facets per query;
- 10 gold chunks per facet;
- 30 distractors per query;
- 10,000–20,000 total chunks;
- top- k , MMR, facility-location at $k \in \{5, 10, 20\}$;
- Facet Coverage@ k , Weighted Facet Coverage@ k , GP, GR, Dominant-Cluster Concentration@ k , and Distractor Rate@ k .

After this works, scale to more conditions and query types.

References

- Zahra Abbasiantaeb, Simon Lupart, and Mohammad Aliannejadi. Generating multi-aspect queries for conversational search. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, pages 8201–8217, 2026. doi: 10.18653/v1/2026.eacl-long.383. URL <https://aclanthology.org/2026.eacl-long.383/>.
- Arzideh et al. Improving retrieval augmented generation for health care by fine-tuning clinical embedding models, 2026.
- Cristian Cosentino, Annamaria Defilippo, Marco Dossena, Christopher Irwin, Sara Joubbi, and Pietro Liò. Healthbranches: Synthesizing clinically-grounded question answering datasets via decision pathways, 2025.
- Ilias Driouich, Hongliu Cao, and Eoin Thomas. Diverse and private synthetic datasets generation for RAG evaluation: A multi-agent framework, 2025. URL <https://arxiv.org/abs/2508.18929>.
- Alvaro Figueira and Bruno Vaz. Survey on synthetic data generation, evaluation methods and gans. *Mathematics*, 10(15):2733, 2022. doi: 10.3390/math10152733.
- Albert Gong, Kamilė Stankevičiūtė, Chao Wan, Anmol Kabra, Raphael Thesmar, Johann Lee, Julius Klenke, Carla P. Gomes, and Kilian Q. Weinberger. PhantomWiki: On-demand datasets for reasoning and retrieval evaluation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL <https://arxiv.org/abs/2502.20377>.
- Lecheng Gong, Weimin Fang, Ting Yang, Dongjie Tao, Chunxiao Guo, Peng Wei, Bo Xie, Jinqun Guan, Zixiao Chen, Fang Shi, Jinjie Gu, and Junwei Liu. Meddialogrubrics: A comprehensive benchmark and evaluation framework for multi-turn medical consultations in large language models, 2026.
- Yue Huang, Siyuan Wu, Chujie Gao, Dongping Chen, Qihui Zhang, Yao Wan, Tianyi Zhou, Chaowei Xiao, Jianfeng Gao, Lichao Sun, and Xiangliang Zhang. DATAGEN: Unified synthetic dataset via large language models. In *International Conference on Learning Representations*, 2025.
- Aryan Jadon, Avinash Patil, and Shashank Kumar. Enhancing domain-specific retrieval-augmented generation: Synthetic data generation and evaluation using reasoning models. In *2025 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology*, pages 445–452, 2025. doi: 10.1109/IAICT65714.2025.11100564.
- Jin et al. Biomedical question answering: A survey of approaches and challenges, 2022.
- Priyanka Kargupta, Runchu Tian, and Jiawei Han. Beyond true or false: Retrieval-augmented hierarchical analysis of nuanced claims. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 29664–29679, 2025. doi: 10.18653/v1/2025.acl-long.1434. URL <https://aclanthology.org/2025.acl-long.1434/>.
- Tristan Kenneweg, Philip Kenneweg, and Barbara Hammer. RAGVAL: Automatic dataset creation and evaluation for RAG systems. In *2024 2nd International Conference on Foundation and Large Language Models*, pages 470–475, 2024. doi: 10.1109/FLLM63129.2024.10852482.
- Seungone Kim, Juyoung Suk, Xiang Yue, Vijay Viswanathan, Seongyun Lee, Yizhong Wang, Kiril Gashteovski, Carolin Lawrence, Sean Welleck, and Graham Neubig. Evaluating language models as synthetic data generators. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 6385–6403, 2025. URL <https://aclanthology.org/2025.acl-long.320/>.

- Jeongsoo Lee, Daeyong Kwon, Kyohoon Jin, Junnyeong Jeong, Minwoo Sim, and Minwoo Kim. MHTS: Multi-hop tree structure framework for generating difficulty-controllable QA datasets for RAG evaluation, 2025. URL <https://arxiv.org/abs/2504.08756>.
- Zhuoyan Li, Hangxiao Zhu, Zhuoran Lu, and Ming Yin. Synthetic data generation with large language models for text classification: Potential and limitations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Junyong Lin, Lu Dai, Ruiqian Han, Yijie Sui, Ruilin Wang, Xingliang Sun, Qinglin Wu, Min Feng, Hao Liu, and Hui Xiong. ScIRGen: Synthesize realistic and large-scale RAG dataset for scientific research. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5619–5630, 2025. doi: 10.1145/3711896.3737432. URL <https://dl.acm.org/doi/10.1145/3711896.3737432>.
- Nikita Mehandru, Niloufar Golchini, Namrata Garg, Kathy T. LeSaint, Christopher J. Nash, Anu Ramachandran, Travis Zack, Liam G. McCoy, Adam Rodman, David Bamman, Melanie Molina, and Ahmed Alaa. ER-Reason: A benchmark dataset for LLM clinical reasoning in the emergency room, 2026.
- Larissa Montenegro, Luis M. Gomes, and José M. Machado. What we know about the role of large language models for medical synthetic dataset generation. *AI*, 6(6):109, 2025. doi: 10.3390/ai6060109.
- Nadas et al. Synthetic data generation using large language models: Advances in text and code, 2025.
- Diego Pastríán, Nicolás Hidalgo, Víctor Reyes, and Erika Rosas. Evolutionary multi-objective prompt learning for synthetic text data generation with black-box large language models. *Applied Sciences*, 16(8):3623, 2026. doi: 10.3390/app16083623. URL <https://doi.org/10.3390/app16083623>.
- Qureshi et al. Improving medical data quality via synthetic data generation, 2026.
- Hossein A. Rahmani, Nick Craswell, Emine Yilmaz, Bhaskar Mitra, and Daniel Campos. Synthetic test collections for retrieval evaluation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2647–2651, 2024. doi: 10.1145/3626772.3657942. URL <https://dl.acm.org/doi/10.1145/3626772.3657942>.
- Hanshu Rao, Weisi Liu, Haohan Wang, I-Chan Huang, Zhe He, and Xiaolei Huang. A scoping review of synthetic data generation by language models in biomedical research and application: Data utility and quality perspectives. *Journal of Healthcare Informatics Research*, 10:367–392, 2026. doi: 10.1007/s41666-026-00229-9.
- Haiyang Shen, Hang Yan, Zhongshi Xing, Mugeng Liu, Yue Li, Zhiyang Chen, Yuxiang Wang, Jiuzheng Wang, and Yun Ma. DRAGON: Domain-specific robust automatic data generation for RAG optimization. In *Findings of the Association for Computational Linguistics: EACL*, pages 1065–1078, 2026. doi: 10.18653/v1/2026.findings-eacl.56. URL <https://aclanthology.org/2026.findings-eacl.56/>.
- Yuhong Sun, Joachim Rahmfeld, Chris Weaver, Weijia Chen, Roshan Desai, Wenxi Huang, and Mark H. Butler. EnterpriseRAG-Bench: A RAG benchmark for company internal knowledge, 2026. URL <https://arxiv.org/abs/2605.05253>.
- Samuel Thio, Matthew Lewis, Spiros Denaxas, and Richard J. B. Dobson. Unlocking electronic health records: A hybrid graph RAG approach to safe clinical AI for patient QA. *Frontiers in Digital Health*, 8:1780700, 2026. doi: 10.3389/fdgth.2026.1780700.

- Villarreal-Zegarra and Bellido-Boza. Generation of synthetic data in health surveys using large language models, 2026.
- Xiang Yue, Xinliang Frederick Zhang, Ziyu Yao, Simon Lin, and Huan Sun. CliniQG4QA: Generating diverse questions for domain adaptation of clinical question answering. In *2021 IEEE International Conference on Bioinformatics and Biomedicine*, pages 580–587, 2021. doi: 10.1109/BIBM52615.2021.9669300.
- Zafar et al. KI-MAG: A knowledge-infused abstractive question answering system in medical domain, 2024.
- Angelo Ziletti and Leonardo D’Ambrosi. Generating patient cohorts from electronic health records using two-step retrieval-augmented text-to-SQL generation, 2025.